

CS/SE/CE 3354 Software Engineering

Team Final Project Specifications (Part IV)

PROJECT DELIVERABLE # 2

Locale

Repository Link: https://github.com/Lysdexi8/3354_Team8.git

Group Members & Task Delegation:

- **Joseph Munoz:** Worked on presentation slides, created prototype, assisted with cost analysis
- **Meher Alapati:** Worked on presentation slides, completed the comparison of work portion
- **Vinh Nguyen:** Worked on presentation slides, completed test case, assisted with cost analysis and conclusion
- **Ujwal Chillakuru:** Worked on presentation slides, assisted with comparison of work, formatting, and citations
- **Anav Bajpai:** Worked on presentation slides and completed most of the conclusion
- **Jason Kam:** Worked on presentation slides, completed project timeline, assisted with cost analysis

Assumptions:

- We assume that third-party services like Google Maps API will be operational, accurate, and accessible for our software to utilize and rely on
- We assume that our software will be developed in the state of Texas, therefore it will comply with any relevant laws like the Texas Data Privacy And Security Act for any current and future development
- We assume that the local services and business will provide accurate and updated information like opening hours, phone number, and website if available

Feedback:

Feedback Received: “The project plan appears to be sound.”

Because our Locale application proposal was approved as being a “sound” and solid plan with no further comments regarding improvements or modification, we will confidently proceed with our original plan and design implementations. Thus:

1. We will implement our mobile service match-making application, Locale, that offers suitable matches (to restaurants, bookstores, healthcare providers, etc.) based on user preferences.
2. To accomplish this, during each project stage, our team will generally work on tasks together but individual members may volunteer or be assigned specific responsibilities that suit their strengths (roles may change).

Software Process Model:

Incremental Development (+ Agile Implementation) -

We chose the incremental development software process model, because it reduces the cost of accommodating customer requirement changes, makes it easier to obtain and apply customer feedback, and deploy software to customers more rapidly. We also plan to implement an Agile approach, because of its iterative nature allowing us to continuously improve its software evolution throughout development.

Instead of trying to develop every possible feature all at once, we can develop our software with the bare minimum functionalities, allowing us to release our software earlier in development. As a result, customers would be able to utilize our software sooner and provide feedback. Using that feedback, we would repeat the software development process with that feedback in mind to add more features, fix issues, etc. Thus reducing the cost of analysis and documentation for changing requirements and enabling our software to evolve to customer needs.

Requirements (Reqs) -

Functional:

1. The system shall filter and display a list of local services that match the user's selected preferences (dietary, accessibility, and location)
2. The system shall allow users to edit and save changes to their user preference
3. The system shall verify user identity and grant them access to their accounts once validated through a registered email address and its associated password in a dedicated logins page
4. The system shall provide administrators a control panel that allows them to add, modify, and remove local services and their accommodation details
5. The system shall allow users to save service providers and businesses to a dedicated "Favorites" section within their profile
6. The system shall display a map view with the physical locations of their matches

7. The system shall display a service's operational details for users to view or interact (opening hours, phone number, and website if available)

Non-Functional:

- **Product Reqs:**
 - **Usability Reqs** - The system shall allow users to search for their desired local services with a minimum of 4 clicks after launching the application
 - **Efficiency Reqs:**
 - **Performance Reqs** - The system shall filter and display user's desired search results in under 5 seconds
 - **Space Reqs** - The system shall use no more than 1GB of user's storage space
 - **Dependability Reqs** - The system shall be available 99.9% of the time for users to use
 - **Security Reqs** - The system shall shall encrypt user's password before storing them into a data repository
- **Organizational Reqs:**
 - **Environmental Reqs** - The system shall adjust the user interface layout to fit various screen sizes, from phones to tablet screens
 - **Operational Reqs** - The system shall will require an active connection in order to perform searches and match-making, and display a "No Network Connection" status otherwise
 - **Developmental Reqs** - The system shall properly implement consistent coding standards and documentation to ensure future maintainability and scalability
- **External Reqs:**
 - **Regulatory Reqs** - The system shall obtain explicit consent from the user before processing any sensitive personal data through a dedicated terms of agreement at initial sign up

- **Ethical Reqs** - The system shall not share or store user information, location, favorite services, and preference to third parties without obtaining user consent
- Legislative Reqs:
 - **Accounting Reqs** - The system shall track, maintain a log, and inform users of all the application versions, modifications, bug fixes, and updates
 - **Safety/Security Reqs** - The system shall allow users to set up two-step authentication of their account

Use Case Diagram:

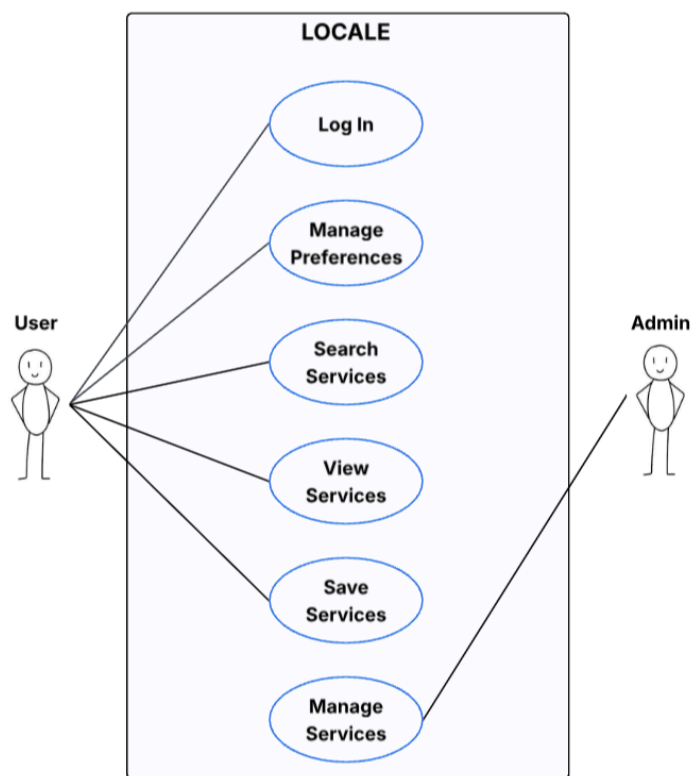


Figure 1. Use Case Diagram for Locale Application

Sequence Diagram:

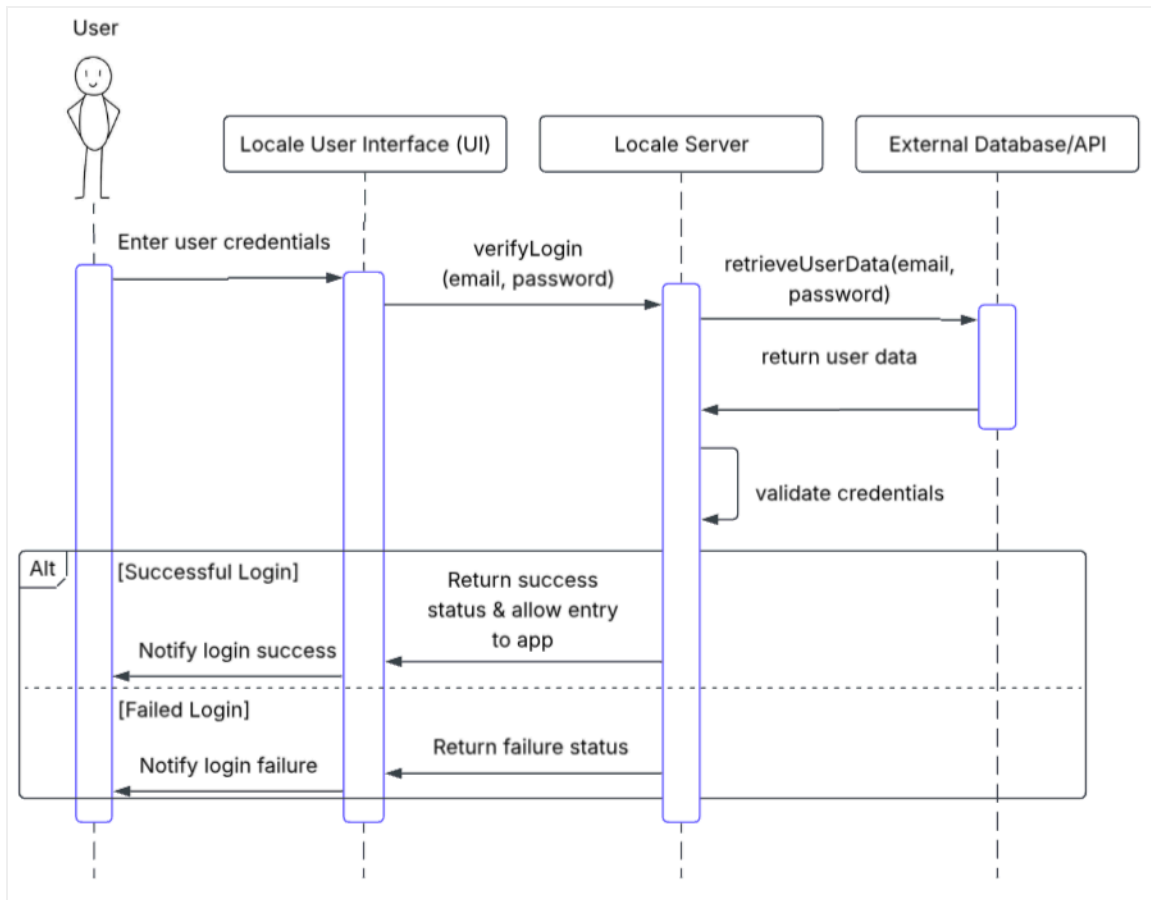


Figure 2. Log In (User Use Case) Sequence Diagram

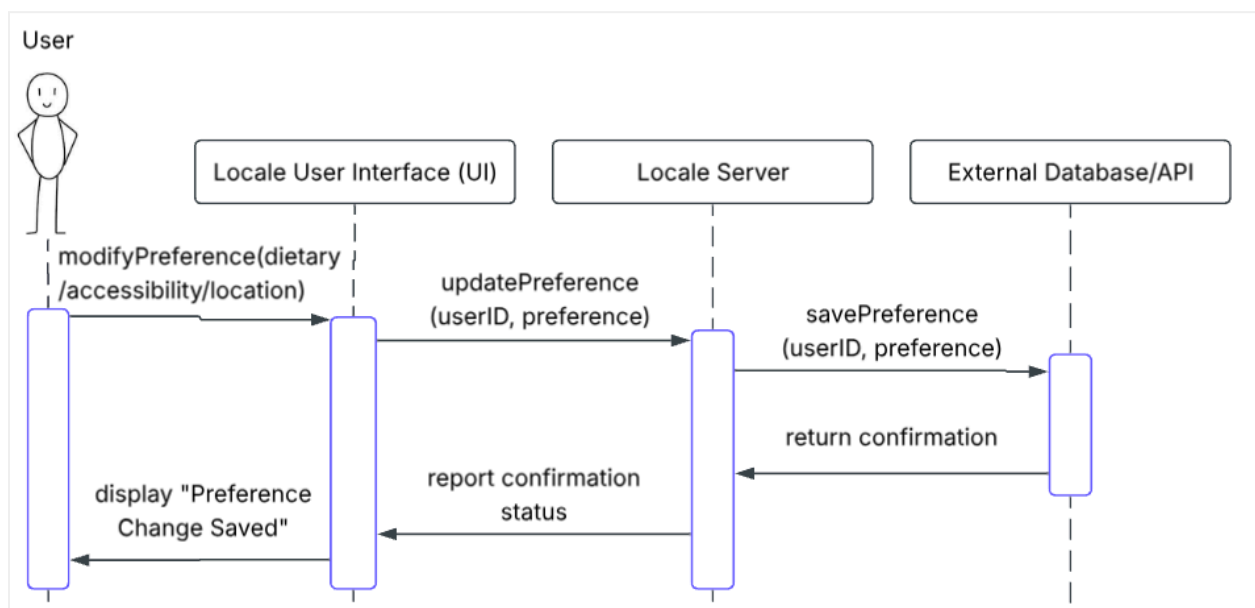


Figure 3. Manage Preference (User Use Case) Sequence Diagram

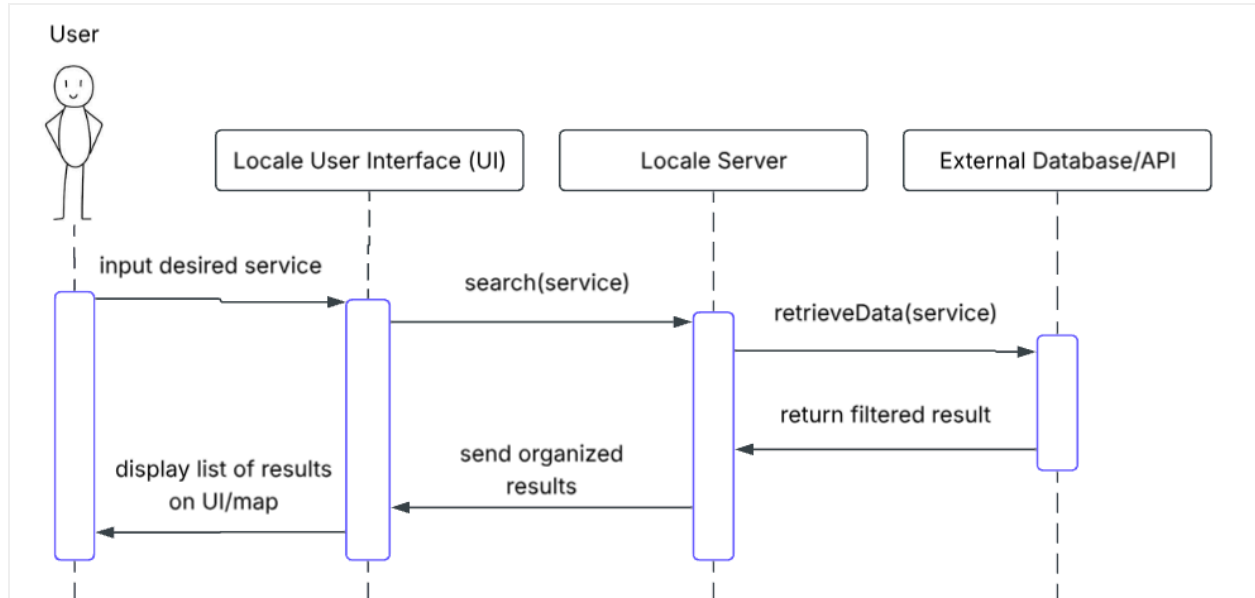


Figure 4. Search Services (User Use Case) Sequence Diagram

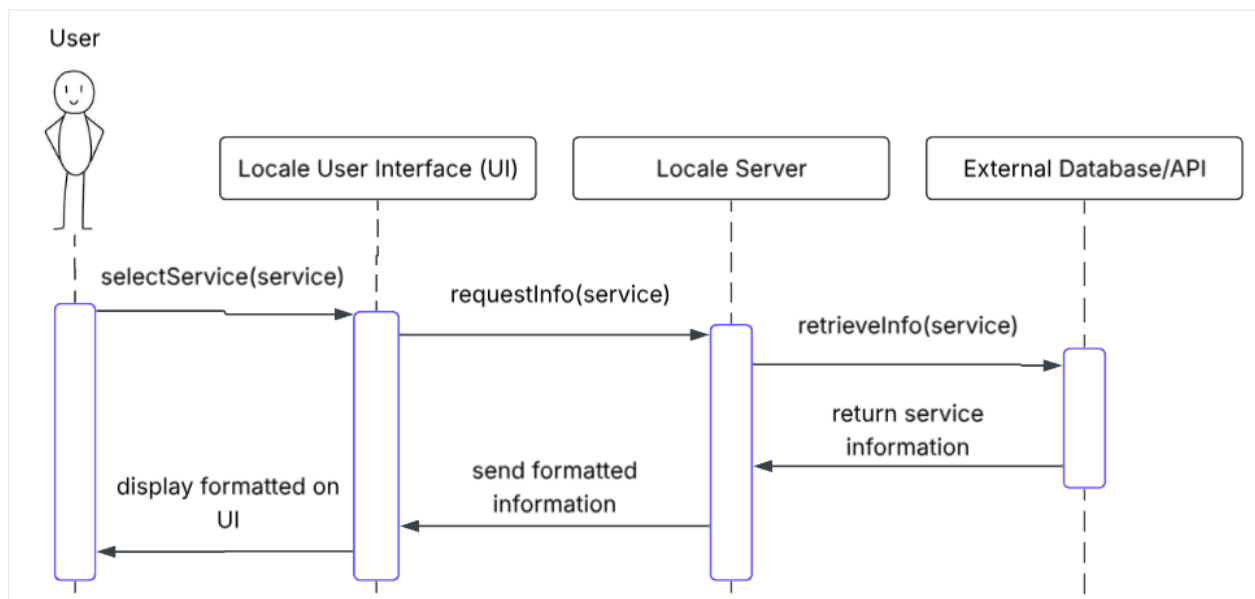


Figure 5. View Services (User Use Case) Sequence Diagram

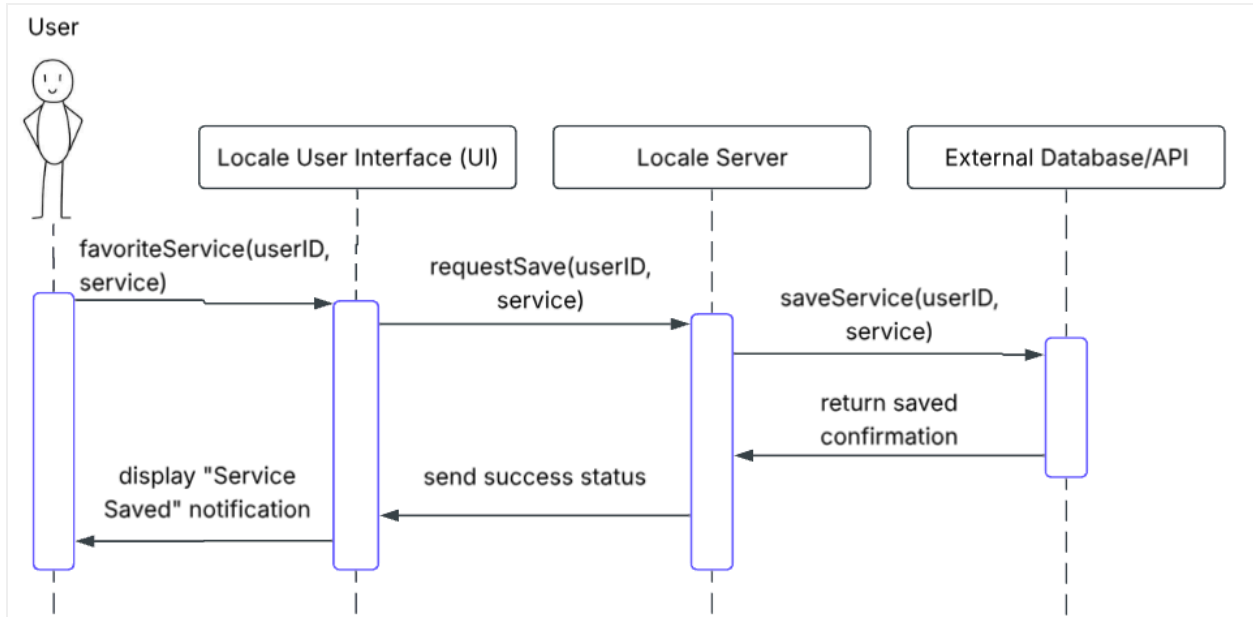


Figure 6. Save Services (User Use Case) Sequence Diagram

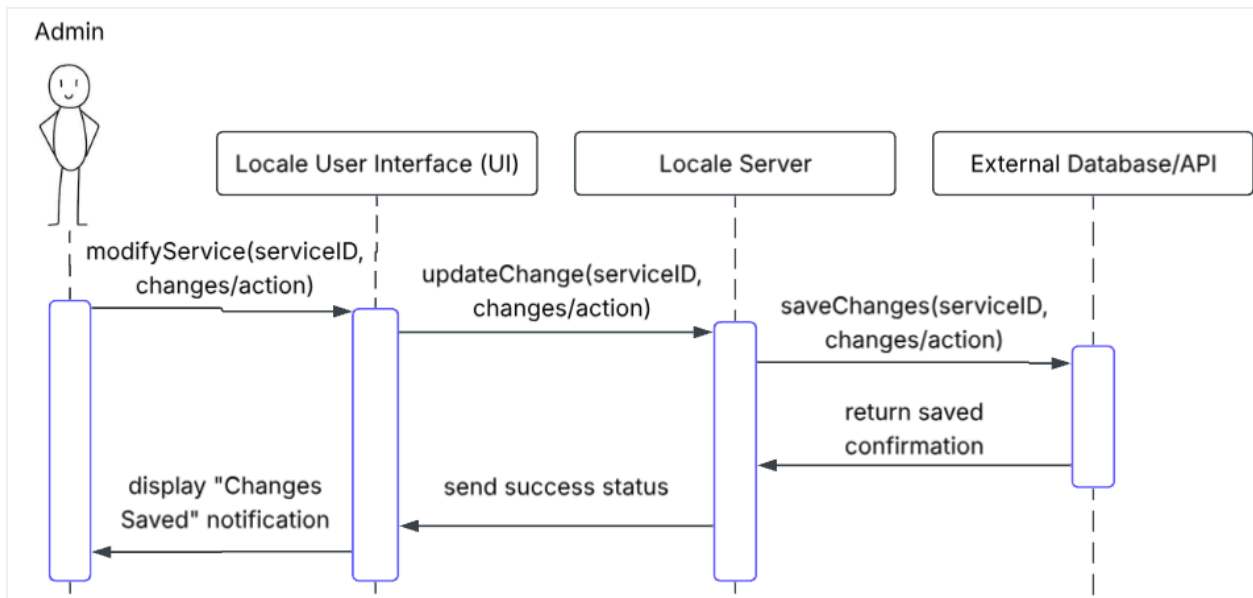


Figure 7. Modify Services (Admin Use Case) Sequence Diagram

Class Diagram:

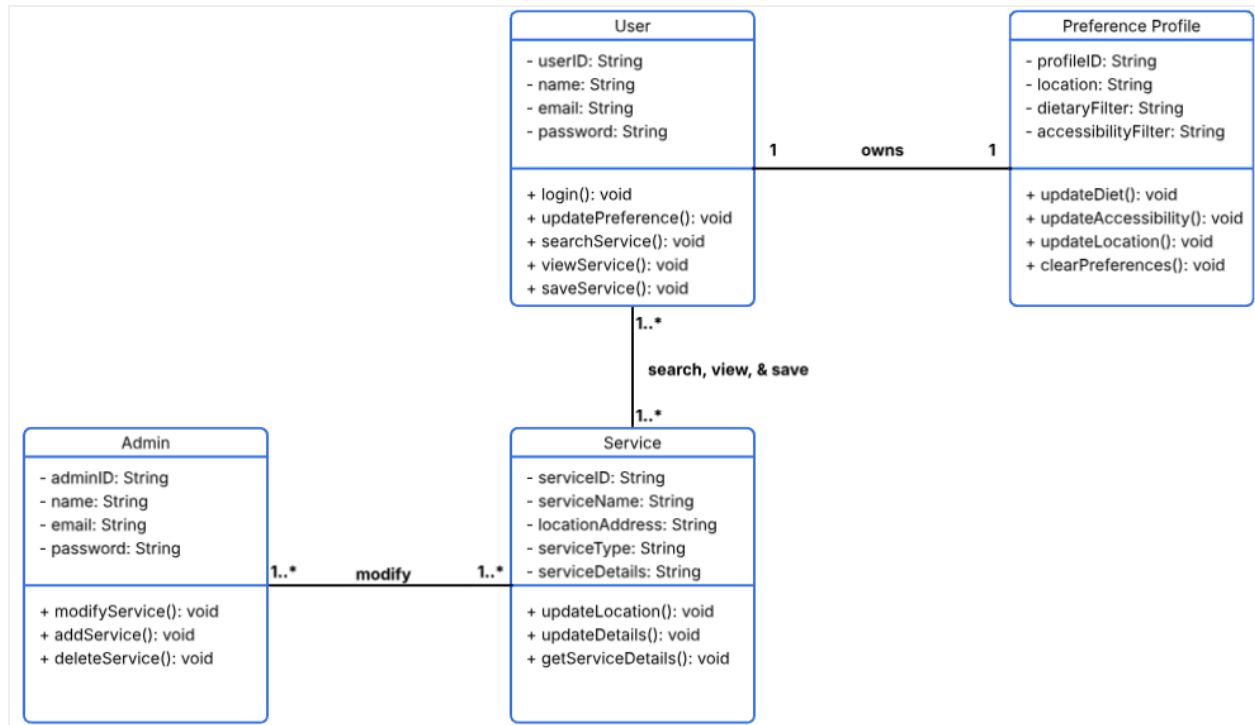


Figure 8. Locale Application Class Diagram

Architectural Diagram:

Client-Server Architectural Pattern -

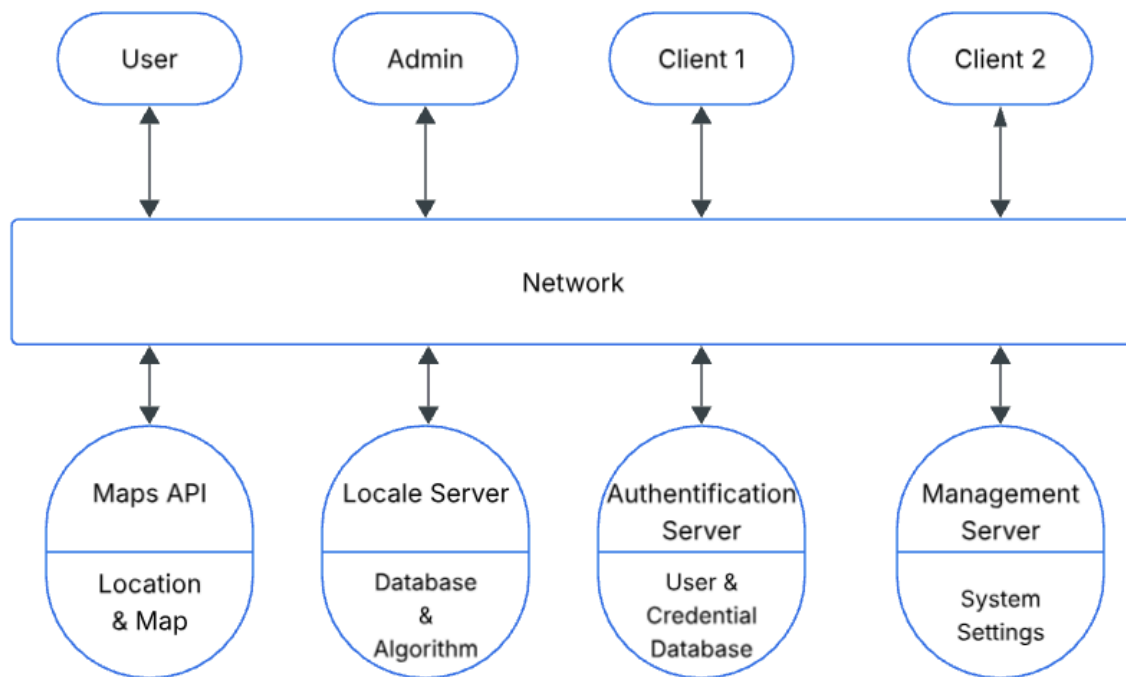


Figure 9. Locale Application Architectural Diagram

Client & Server architecture would be the most appropriate architectural pattern for our preference match-making mobile application team project. The mobile application acts as the “Client”, displaying and conveying user preferences and matches, while the “Server” is the backend, storing and handling all the information of all these services (libraries, restaurants, healthcare providers, etc.), match-making algorithm, and user preference data. Client-Server architecture provides reliable access to these services across different locations which is essential to our application. Additionally, it would make development efficient since basic functionalities would be available to all clients without having to implement them individually to every potential user.

Project Scheduling:

- **Project Start Date:** January 20, 2027
- **Project End Date:** June 20, 2027 (6 months / 24 weeks)
- **Justification:** A 6-month development lifecycle was selected to properly utilize the Incremental Development and Agile software process model in developing Locale. This half-year time period is sufficient enough to accommodate the architectural and planning phase, backend and API integration phase, frontend and UI implementation phase, agile iteration and performance optimization phase, security and compliance testing phase, and finally deployment phase. It's enough time to utilize Client-Server architecture and accommodate the implementation, testing, and potential errors that come with relying on third-party services.
- **Work Schedule:**
 - Weekends will NOT be counted in the schedule
 - 8 Working Hours per Full-Time Developer (40 Hours / Week)

Cost, Effort and Pricing Estimation:

- **Cost Modeling Technique:** Function Point (FP)
- **External Inputs (EI):** User Login, User Preference, Admin Service Modifications (3 *total*)
- **External Outputs (EO):** Search Results, Map View, Confirmation Alerts/Messages (3 *total*)
- **External Queries (EQ):** Favorites List, Service Details (2 *total*)
- **Internal Logical Files (ILF):** User Preference Data, User Account Data, Local Service Database (3 *total*)
- **External Interface Files (EIF):** Google Maps API, Authentication Service Database (2 *total*)

Component	Type	FTRs/RETs	Data Element	Complexity
-----------	------	-----------	--------------	------------

User Login	External Input (EI)	1 FTR	2 DETs (Email & Password)	Low
User Preference	External Input (EI)	1 FTR	3 DETs (Dietary, Accessibility, Location)	Low
Admin Service Modifications	External Input (EI)	2 FTRs	6 DETs (Name, service ID, Address, Type, Details, Submission)	Average
Search Results	External Output (EO)	2 FTRs	6 DETs (Name, Address, Service Type, Operational Details, Distance, Information Details)	Average
Map View	External Output (EO)	2 FTRs	4 DETs (User Pin, Service Pin, Map UI, Distance)	Low
Confirmation Alerts/Messages	External Output (EO)	1 FTR	2 DETs (Status, Message)	Low
Favorites List	External Queries (EQ)	2 FTRs	5 DETs (Name, Address, Type, Details, Favorite Status)	Low
Service Details	External Queries (EQ)	1 FTR	8 DETs (Service ID, Name, Address, Location, Type, Information, Contact, Website, Save Status)	Low
User Preference Data	Internal Logical Files (ILF)	1 RET	4 DETs (Profile ID, Location, Dietary Filter,	Low

			Accessibility Filter)	
User Account Data	Internal Logical Files (ILF)	1 RET	5 DETs (User ID, Name, Email, Password, Phone Number)	Low
Local Service Database	Internal Logical Files (ILF)	2 RETs	9 DETs (Service ID, Name, Location, Type, Details, Operational Hours, Contact, Website, Map View)	Low
Google Maps API	External Interface Files (EIF)	1 RET	4 DETs (Address, Latitude Coordinates, Longitude Coordinates, Distance)	Low
Authentication Service Database	External Interface Files (EIF)	1 RET	2 DETs (Encrypted Credential Token & Validation Status Token)	Low

Table 1. Determining Complexity of Components

Component Type	Component Complexity			Total
	Low	Average	High	
External Inputs (EI)	3 x 2 (counts) = 6	4 x 1 (count) = 4	6 x 0 = 0	10
External Outputs (EO)	4 x 2 (counts) = 8	5 x 1 (count) = 5	7 x 0 = 0	13
External	3 x 2 (counts) =	4 x 0 = 0	6 x 0 = 0	6

Queries (EQ)	6			
Internal Logical Files (ILF)	7 x 3 (counts) = 21	10 x 0 = 0	15 x 0 = 0	21
External Interface Files (EIF)	5 x 2 (counts) = 10	7 x 0 = 0	10 x 0 = 0	10
			Total	60 UFP

Table 2. UFP Table Calculation

General Systems Characteristics	Rating (0-5)
Data Communications	4
Distributed Data Processing	3
Performance	4
Heavily Used Configuration	2
Transaction Rate	3
Online Data Entry	5
End-User Efficiency	4
Online Update	3
Complex Processing	2
Reusability	2
Installation Ease	2
Operational Ease	3
Multiple Sites	0
Facilitate Change	3
Total:	40

Table 3. General Systems Characteristics Rating

Final Function Point (FP) = 60 UFP x (0.65 + 0.01 x 40) → 63 FP [2]

Exponents for Jones's first-order estimate

Kind of software	Best in class	Average	Worst in class
Systems	0.43	0.45	0.48
Business	0.41	0.43	0.46
Shrink-wrap	0.39	0.42	0.45

Table 4. Estimation for Optimal Schedule Months [1]

Schedule Months: $63^{0.43} = \underline{5.47 \text{ months}}$ (justifies estimated 6 month project schedule for best and worst cases)

Effort: $63^{(3 \times 0.43)} / 27 = \underline{7.47 \text{ man-months}}$

- **Cost of Personnel:**

- **Estimated Personal Cost:** 7.47 (effort) x 10954 (average monthly software developer salary) = **\$81,826**

- **Cost of Hardware Products:**

- Workstation: \$6,000
- Testing Devices: \$1,500
- **Total Estimated Hardware Products Cost:** **\$7,500**

- **Cost of Software Products:**

- Database: \$2,000
- Google Maps API: \$1,500
- Licenses: \$2,400
- Developer Fees: \$200
- **Total Estimated Hardware Products Cost:** **\$6,100**

Total Development Cost: $81,826 + 7,500 + 6,100 = \underline{\underline{\$95,426}}$

Test Plan:

For the Locale application, we will utilize Unit Testing, which will allow us to isolate and test components individually. More specifically, we use Automated Tests and automated frameworks like JUnit so that tests can be run and checked without manual intervention in order to ensure our code is maintained and robust. For instance, authentication functionality like the verifyLogin

method can be tested to ensure that the software successfully logs in valid users or to help uncover defects that accept invalid credentials.

Unit Testing:

```
package unittestcode;
import java.util.HashMap;
import java.util.Map;

public class UserAuthentication {
    private Map<String,String> userDatabase = new HashMap<>();

    public UserAuthentication() {
        userDatabase.put("chikawa@gmail.com", "white123");
        userDatabase.put("usagi@gmail.com", "yellow123");
        userDatabase.put("hachiware@gmail.com", "blue123");
    }

    public boolean verifyLogin(String email, String password) {
        if (userDatabase.containsKey(email)) {
            if (password.equals(userDatabase.get(email))) {
                System.out.println("Login Successful");
                return true;
            } else {
                System.out.println("Invalid password");
                return false;
            }
        }
        System.out.println("Invalid email");
        return false;
    }
}
```

Figure 10. Implementation Code for verifyLogin Method Example

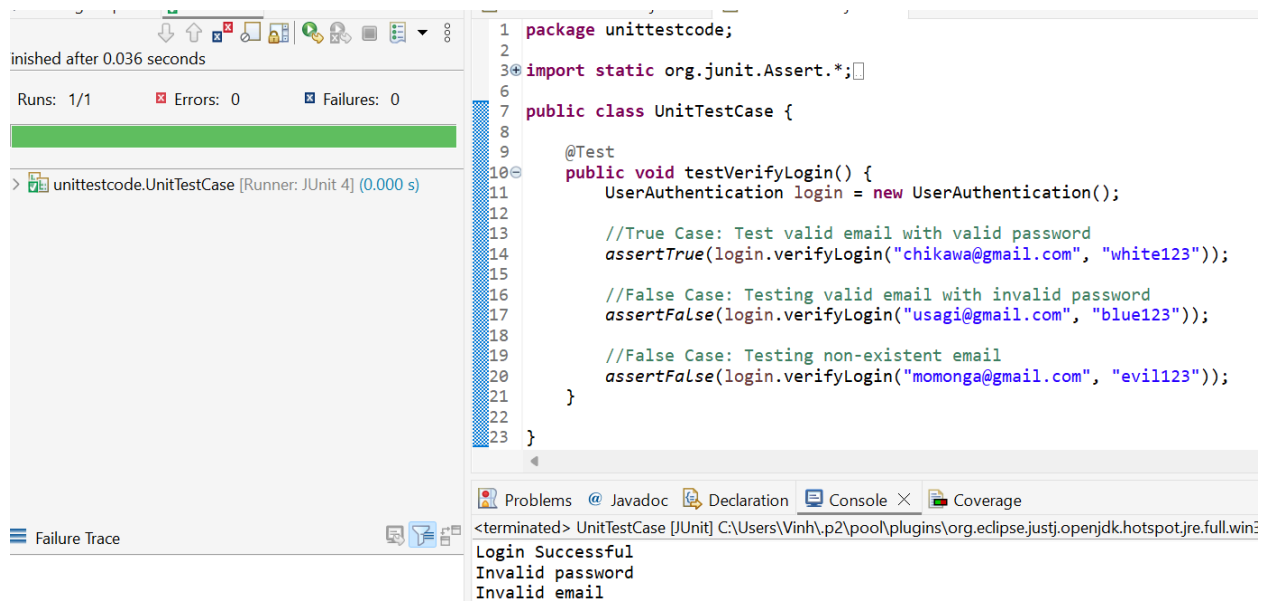


Figure 11. JUnit Execution Results

Test Cases:

- **Valid Login (Validation Test) -**
 - To demonstrates software successfully logs in valid users
 - Email: “chikawa@gmail.com” and Password: “white123” where BOTH do exist in the database
 - This test case validates whether verifyLogin method returns true
 - **Results Obtained:** The test case (assertTrue) evaluated to true because the verifyLogin returned true and outputted the successful message “Login Successful”. Thus this test case passed without error.
- **Invalid Password (Defect Test) -**
 - To uncover defects in software to accurately deny login to invalid passwords even if the email is valid
 - Email: “usagi@gmail.com” and Password: “blue123” where BOTH do exist in the database but the password does not correspond with the email
 - This test case validates whether verifyLogin method returns false
 - **Results Obtained:** The test case (assertFalse) evaluated to true because the verifyLogin returned false and outputted the unsuccessful message “Invalid Password”. Thus this test case passed without error.
- **Invalid Email (Defect Test)**
 - To ensure the software accurately denies login attempt to non-existent users or email accounts
 - Email: “momonga@gmail.com” and Password: “evil123” where email does NOT exist in the database
 - This test case validates whether verifyLogin method returns false
 - **Results Obtained:** The test case (assertFalse) evaluated to true because the verifyLogin returned false and outputted the unsuccessful message “Invalid Email”. Thus this test case passed without error.

Comparison of Work:

To evaluate our proposed application, we compared Locale with several existing location-based applications including Google Maps, Yelp, and TripAdvisor. These applications are widely used by many for comparison, finding restaurants, businesses, and other local services. While providing useful search recommendation features, they are designed for more general use and are not focused on matching services based on a user’s preferences and needs.

Google Maps offers one of the largest databases of businesses and services and provides accurate navigation, business information, customer reviews, and operating hours. However, its recommendations are mostly based on location, popularity, and previous activity rather than detailed user needs. Although Google Maps allows users to search using keywords, the

application does not allow them to create a profile with specific dietary, accessibility, or lifestyle preferences that influence every recommendation.

Yelp is another popular application that focuses on reviews and recommendations for restaurants and local businesses. It allows users to filter search results by certain features such as price range per person or dietary restrictions and provides detailed customer reviews. However, Yelp mainly focuses on restaurants and entertainment-related businesses, making it less suitable for users looking for other services such as healthcare providers or bookstores.

Compared to these existing applications, Locale combines features from multiple platforms while placing a stronger emphasis on personalization. Instead of only displaying nearby businesses, Locale recommends services based on a user's saved preferences, including dietary restrictions, accessibility needs, location, and favorite categories. By supporting multiple types of services within a single application and using preference-based matching, Locale aims to make it easier for users to quickly find businesses that fit their individual needs rather than relying solely on general search.

Feature	Google Maps	Yelp	TripAdvisor	Locale
Personalized recommendations	Limited	Limited	Limited	Yes
Restaurant recommendations	Yes	Yes	Yes	Yes
Healthcare providers	Yes	No	Limited	Yes
Bookstores & other services	Yes	Limited	Limited	Yes
Dietary preference filtering	Partial	Partial	No	Yes
Accessibility preference filtering	Limited	Limited	No	Yes
Favorites	Yes	Yes	Yes	Yes
User preference profile	Limited	Limited	Limited	Yes

Table 5. Existing Services Comparison

Conclusion:

Although there wasn't much we needed to change with our work because our initial project proposal was approved without modification, we did not need to deviate from our originally planned project design/scope. However, there were definitely some things we could have

improved on after our first deliverable. For instance, using cookies to remember which user last logged into the app, to streamline user experience, instead of having them log in every single time. This would work well with the usability requirements of utilizing 2-Factor Authentication to provide a convenient yet secure way to log into Locale. Additionally, we could have included third parties of interest like the service providers themselves as a stakeholder with the ability to access and potentially alter services (as a creator) through an approval process since they are the ones that know their service best, especially the dietary or accommodations that our mobile application is mainly about, which would streamline the backend process to avoid having to use a third party service to obtain the service details when we could get it directly from the service providers themselves.

Throughout this project, our team designed Locale, a mobile application that provides personalized recommendations for restaurants, bookstores, healthcare providers, and local services. Based on individual user preferences. By applying software engineering principles such as requirement analysis, UML diagrams, architectural design, project scheduling, cost estimation and software testing, we developed a robust, well-rounded design for our proposed app Locale.

The main challenges we encountered during this project was selecting the appropriate software development methodologies to determine which best fit our project platform (as it is a mobile application) and objective. It was especially important to ensure that the chosen methodology could evolve without requiring revision of previous development and documentation as possible. Another challenge was identifying system's functionalities especially when the scope of the project and restraints were assumed and uncertain while also trying to maintain as detailed and extensive as possible. Creating diagrams also posed a challenge, especially when trying to convey clear and accurate representation of the software system, stakeholders, and the relationship between them.

Some lessons learned were the importance of initial planning. Initial planning sets the foundation for the project while also defining the goals for the project to ensure we do not deviate from what was originally planned. Additionally, communication is key. Lack of communication leads to disorganization where developers are not on the same page and deadlines are pushed to their limit which increases development risk, likelihood of failure, and incomplete projects.

This project shows the importance of selecting the software process method, and early gathering of project requirements and constraints. Drawing diagrams/defining a realistic scope and features are also good for ensuring project deliverability. Overall, the experience shows the value of planning, selecting the right methodology, and documentation in software engineering projects. Any other projects should also use these lessons to reduce risks and enhance quality.

References:

- [1] F. Swartz, “Software Engineering: Function Point Analysis - Effort Estimation,” 2026. Accessed: July 26, 2026. [Online]. Available: https://www.cs.rutgers.edu/courses/111/classes/fall_2011_tjang/texts/notes-java/principles_and_practices/fpa/fpa-effort.html
- [2] GeeksforGeeks, “Functional Point (FP) Analysis Software Engineering,” Apr. 2019. Accessed: July 26, 2026. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/software-engineering-functional-point-fp-analysis/>
- [3] Tripadvisor, “US Press Center | Resources,” *MediaRoom*, 2025. <https://tripadvisor.mediaroom.com/us-resources> (accessed July 26, 2026).
- [4] U.S. BUREAU OF LABOR STATISTICS, “Software Developers, Quality Assurance Analysts, and Testers,” U.S. Bureau of Labor Statistics, Apr. 2025. Accessed: July 26, 2026. [Online]. Available: <https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>
- [5] Yelp, “About Us | Yelp,” 2019. Accessed: July 26, 2026. [Online]. Available: <https://www.yelp.com/about>